

Détection de *faux billets*

Un algorithme de classification pour l'ONCFM

— Christelle Laval — OPENCLASSROOMS

CONTEXTE

Contexte & problématique

POURQUOI CE PROJET

- L'ONCFM souhaite automatiser la détection des faux billets en euros à partir de mesures géométriques.
- Aujourd'hui, l'authentification repose sur un contrôle qui ne peut pas être généralisé à grande échelle.
- Objectif : construire un classifieur binaire vrai / faux à partir de 6 variables géométriques mesurées par une machine dédiée.

CONTRAINTE MÉTIER MAJEURE

Un faux billet classé comme vrai — un **faux négatif** — est l'erreur la plus coûteuse pour l'organisation.

DONNÉES

Le jeu de données — billets.csv

RÉPARTITION — 1 500 BILLETS



0
doubleton détecté

2,5 %
valeurs manquantes (margin_low)

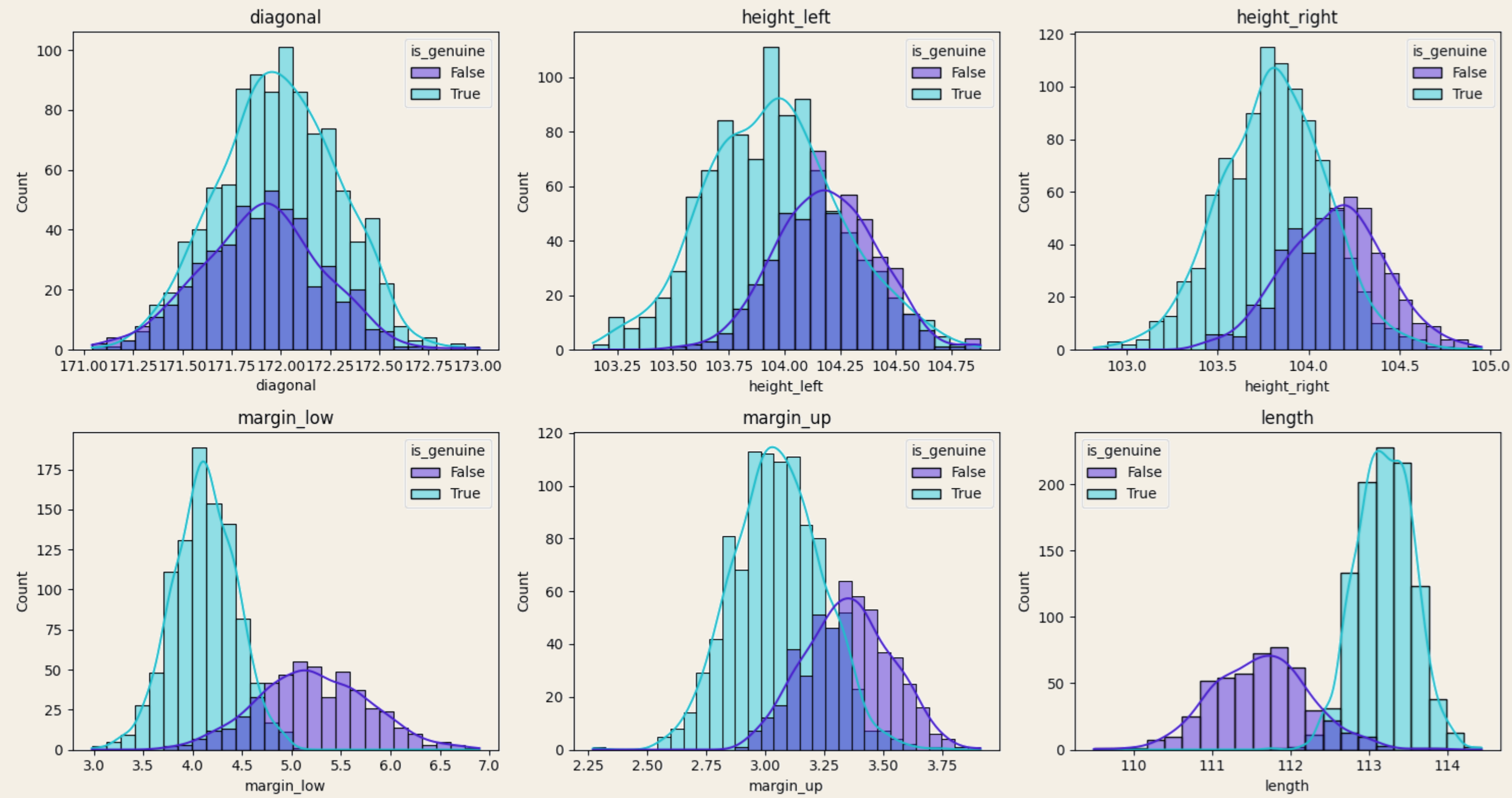
6 VARIABLES GÉOMÉTRIQUES (MM)

- diagonal
- height_left
- height_right
- Margin_low
- margin_up
- length

Cible : `is_genuine` (True = vrai billet)

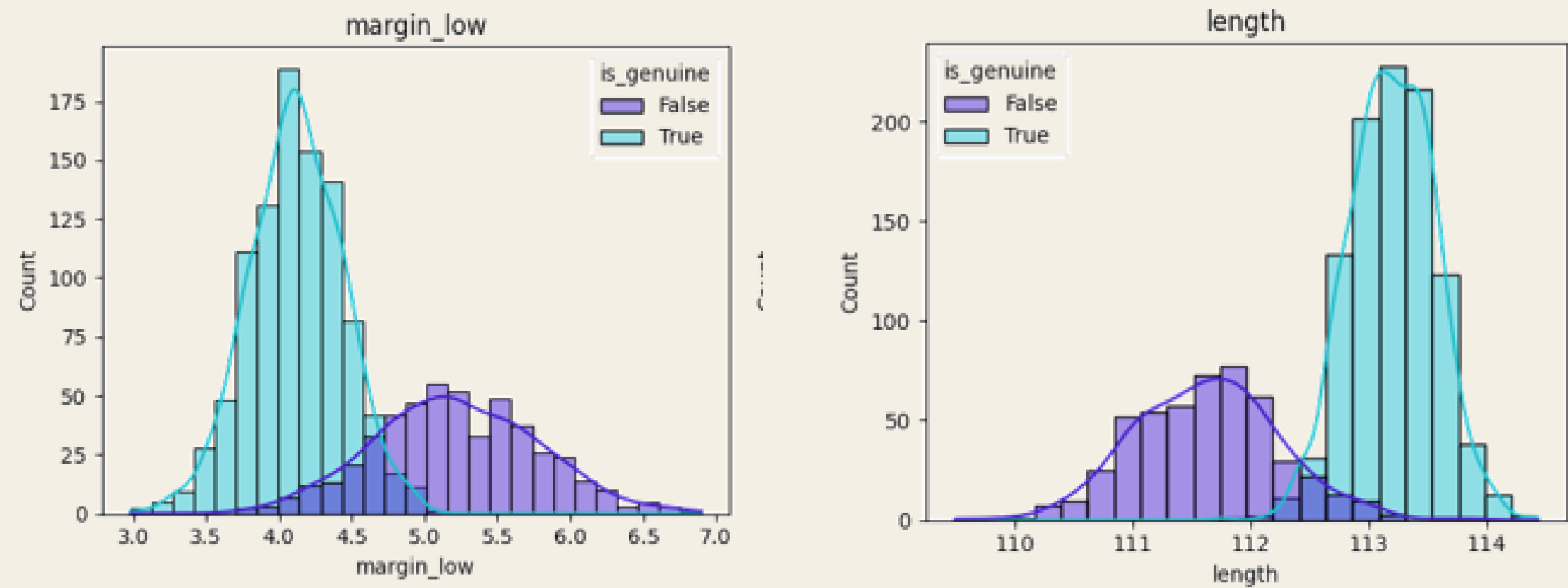
Analyse descriptive

Distribution des 6 variables selon vrai / faux



Analyse descriptive

Distribution des 6 variables selon vrai / faux



margin_low et **length** séparent le mieux visuellement les deux classes.

MOINS DISCRIMINANTES

- diagonal
- height_left
- height_right
- margin_up

Traitement des valeurs manquantes

margin_low — 37 valeurs, 2,5 %

CHOIX RETENU

Imputation par régression linéaire à partir des 5 autres variables géométriques.

Corrélation la plus forte avec **length** (−0,67) et **margin_up** (+0,43).

Pourquoi pas dropna() ?

Perte de 37 billets, potentiellement des faux — classe déjà minoritaire.

Pénaliserait l'entraînement sur le critère de rappel visé.

Pourquoi pas une médiane ?

Ignore la corrélation entre margin_low et les autres mesures.

La regression linéaire réduit l'erreur d'un tiers par rapport à la médiane,

	MAE	RMSE
Médiane	0.571	0.752
Régression linéaire	0.383	0.495

Méthodologie — préparation & modèles comparés

Préparation des données

Split train / test : 80 % / 20 %, stratifié

1 200 billets en train, 300 en test

Standardisation (StandardScaler) pour K-means et KNN

4 modèles

Régression logistique

K-means (centroïdes → prédiction)

KNN

Random Forest

Critère de sélection

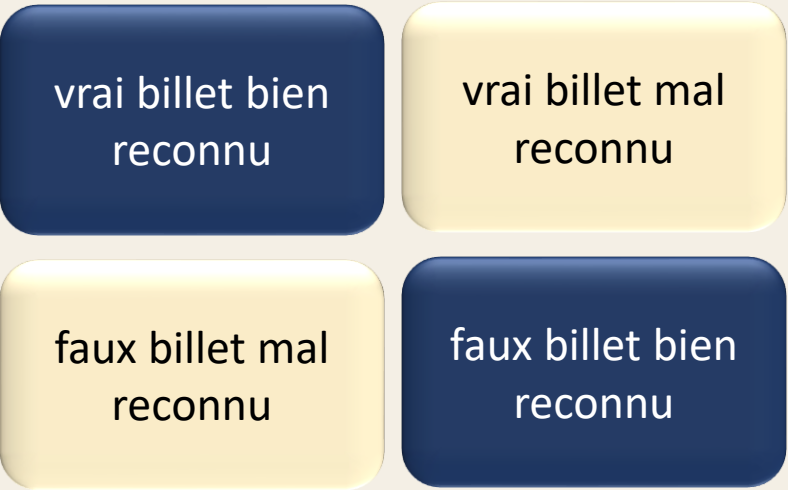
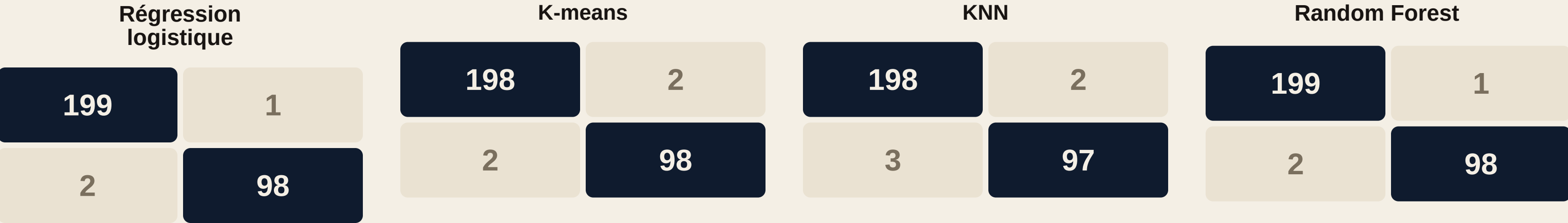
Matrice de confusion + precision / recall / f1 par classe, pour chaque modèle.

Le rappel sur la classe « faux billet » est le critère décisif — pas l'accuracy globale.

RÉSULTATS

Résultats — matrices de confusion

Jeu de test (300 billets)



SYNTHÈSE

Tableau de synthèse

Trié par rappel décroissant sur la classe « faux »

Modèle	Rappel faux	Précision faux	F1 faux	Rappel vrai	Accuracy
Régression logistique — retenu	0,98	0,99	0,99	0,99	0,99
Random Forest	0,98	0,99	0,99	0,99	0,99
K-means	0,98	0,98	0,98	0,99	0,99
KNN	0,97	0,98	0,97	0,99	0,98

Rappel faux — parmi tous les vrais faux billets, la part détectée par le modèle. Le critère prioritaire ici : un faux billet non détecté (faux négatif) est l'erreur la plus coûteuse pour l'ONCFM.

Précision faux — parmi tous les billets *prédits* faux, la part qui l'est vraiment. Mesure les fausses alertes (un vrai billet signalé à tort).

F1 faux — moyenne (harmonique) de rappel faux et précision faux. Résume les deux en un chiffre, pénalise fortement si l'un des deux est faible.

Rappel vrai — parmi tous les vrais billets, la part reconnue comme telle. Symétrique du rappel faux, côté vrais billets.

Accuracy — part de toutes les prédictions (vrais + faux confondus) correctes sur l'ensemble du jeu de test. Vue globale, mais peut masquer un déséquilibre entre les classes (moins fiable seule que le rappel faux ici).

DÉCISION

Modèle final retenu — *Régression logistique*

0,98

Rappel « faux billet »

0,99

Précision « faux billet »

0,99

Accuracy globale

2 faux billets sur 100 sont mal classés comme vrais — à égalité stricte avec la Random Forest sur les 5 métriques du tableau de synthèse.

À métriques équivalentes, la régression logistique est retenue pour sa **simplicité** et son **interprétabilité** — un atout pour expliquer les décisions à l'ONCFM.

LIVRABLE

predict.py

Charge le modèle final déjà entraîné (`model_final.pkl`) — aucun réentraînement à l'exécution.

Deux modes d'entrée : les 6 valeurs d'un billet unique, ou un CSV contenant plusieurs billets.

Réapplique automatiquement l'imputation par régression si `margin_low` est manquant.

```
$ python predict.py --csv billets_production.csv
```

```
billet_1 → variables + label prediction + Valeur
```

```
billet_2 → variables + label prediction + Valeur
```

```
billet_3 → variables + label prediction + Valeur
```

Script autonome, prêt à recevoir de nouvelles données de production.